17 Barriers

17.1 Introduction

Imagine you are writing the graphical display for a computer game. Your program prepares a sequence of *frames* to be displayed by a graphics package (perhaps a hardware coprocessor). This kind of program is sometimes called a *soft real-time* application: real-time because it must display at least 35 frames per second to be effective, and soft because occasional failure is not catastrophic. On a single-threaded machine, you might write a loop like this:

```
while (true) {
  frame.prepare();
  frame.display();
  }
```

If, instead, you have n parallel threads available, then it makes sense to split the frame into n disjoint parts, and to have each thread prepare its own part in parallel with the others.

```
int me = ThreadID.get();
while (true) {
  frame[me].prepare();
  frame[me].display();
 }
```

The problem with this approach is that different threads will require different amounts of time to prepare and display their portions of the frame. Some threads might start displaying the ith frame before others have finished the $(i-1)^{st}$.

To avoid such synchronization problems, we can organize computations such as this as a sequence of *phases*, where no thread should start the ith phase until the others have finished the $(i-1)^{st}$. We have already seen this phased computation pattern before. In Chapter 12, the sorting network algorithms required each comparison phase to be separate from the others. Similarly, in the sample sorting

The Art of Multiprocessor Programming. DOI: 10.1016/B978-0-12-397337-5.00017-4
Copyright © 2012 by Elsevier Inc. All rights reserved.

```
1  public interface Barrier {
2    public void await();
3  }
```

Figure 17.1 The `Barrier` interface.

```
1  private Barrier b;
2   ...
3   while (true) {
4      frame[my].prepare();
5      b.await();
6      frame[my].display();
7  }
```

Figure 17.2 Using a barrier to synchronize concurrent displays.

algorithm, each phase had to make sure that prior phases had completed before proceeding.

The mechanism for enforcing this kind of synchronization is called a *barrier* (Fig. 17.1). A barrier is a way of forcing asynchronous threads to act almost as if they were synchronous. When a thread finishing phase i calls the barrier's `await()` method, it is blocked until all n threads have also finished that phase. Fig. 17.2 shows how one could use a barrier to make the parallel rendering program work correctly. After preparing frame i, all threads synchronize at a barrier before starting to display that frame. This structure ensures that all threads concurrently displaying a frame display the same frame.

Barrier implementations raise many of the same performance issues as spin locks in Chapter 7, as well as some new issues. Clearly, barriers should be fast, in the sense that we want to minimize the duration between when the last thread reaches the barrier and when the last thread leaves the barrier. It is also important that threads leave the barrier at roughly the same time. A thread's *notification time* is the interval between when some thread has detected that all threads have reached the barrier, and when that specific thread leaves the barrier. Having uniform notification times is important for many soft real-time applications. For example, picture quality is enhanced if all portions of the frame are updated at more-or-less the same time.

17.2 Barrier Implementations

Fig. 17.3 shows the `SimpleBarrier` class, which creates an `AtomicInteger` counter initialized to n, the barrier size. Each thread applies `getAndDecrement()` to lower the counter. If the call returns 1 (Line 10), then that thread is the last to reach the barrier, so it resets the counter for the next use (Line 11). Otherwise,

```
1  public class SimpleBarrier implements Barrier {
2    AtomicInteger count;
3    int size;
4    public SimpleBarrier(int n){
5      count = new AtomicInteger(n);
6      size = n;
7    }
8    public void await() {
9      int position = count.getAndDecrement();
10     if (position == 1) {
11       count.set(size);
12     } else {
13       while (count.get() != 0){};
14     }
15   }
16 }
```

Figure 17.3 The `SimpleBarrier` class.

the thread spins on the counter, waiting for the value to fall to zero (Line 13). This barrier class may look like it works, but it does not.

Unfortunately, the attempt to make the barrier reusable causes it to break (see Fig. 17.2). Suppose there are only two threads. Thread *A* applies `getAndDecrement()` to the counter, discovers it is not the last thread to reach the barrier, and spins waiting for the counter value to reach zero. When *B* arrives, it discovers it is the last thread to arrive, so it resets the counter to *n* in this case 2. It finishes the next phase and calls `await()`. Meanwhile, *A* continues to spin, and the counter never reaches zero. Eventually, *A* is waiting for phase 0 to finish, while *B* is waiting for phase 1 to finish, and the two threads starve.

Perhaps the simplest way to fix this problem is just to alternate between two barriers, using one for even-numbered phases, and another for odd-numbered ones. However, such an approach wastes space, and requires too much book-keeping from applications.

17.3 Sense-Reversing Barrier

A *sense-reversing* barrier is a more elegant and practical solution to the problem of reusing barriers. As depicted in Fig. 17.4, a phase's *sense* is a Boolean value: *true* for even-numbered phases and *false* otherwise. Each `SenseBarrier` object has a Boolean `sense` field indicating the sense of the currently executing phase. Each thread keeps its current sense as a thread-local object (see Pragma 17.3.1). Initially the barrier's `sense` is the complement of the local sense of all the threads. When a thread calls `await()`, it checks whether it is the last thread to decrement the counter. If so, it reverses the barrier's sense and continues.

```
1  public SenseBarrier(int n) {
2    count = new AtomicInteger(n);
3    size = n;
4    sense = false;
5    threadSense = new ThreadLocal<Boolean>() {
6      protected Boolean initialValue() { return !sense; };
7    };
8  }
9  public void await() {
10   boolean mySense = threadSense.get();
11   int position = count.getAndDecrement();
12   if (position == 1) {
13     count.set(size);
14     sense = mySense;
15   } else {
16     while (sense != mySense) {}
17   }
18   threadSense.set(!mySense);
19 }
```

Figure 17.4 The `SenseBarrier` class: a sense-reversing barrier.

Otherwise, it spins waiting for the barrier's `sense` field to change to match its own local sense.

Decrementing the shared counter may cause memory contention, since all the threads are trying to access the counter at about the same time. Once the counter has been decremented, each thread spins on the `sense` field. This implementation is well suited for cache-coherent architectures, since threads spin on locally cached copies of the field, and the field is modified only when threads are ready to leave the barrier. The `sense` field is an excellent way of maintaining a uniform notification time on symmetric cache-coherent multiprocessors.

Pragma 17.3.1. The constructor code for the sense-reversing barrier, shown in Fig. 17.4, is mostly straightforward. The one exception occurs on lines 5 and 6, where we initialize the thread-local `threadSense` field. This somewhat complicated syntax defines a thread-local Boolean value whose initial value is the complement of the `sense` field's initial value. See Appendix A.2.4 for a more complete explanation of thread-local objects in Java.

17.4 Combining Tree Barrier

One way to reduce memory contention (at the cost of increased latency) is to use the combining paradigm of Chapter 12. Split a large barrier into a tree of smaller

```
1  public class TreeBarrier implements Barrier {
2    int radix;
3    Node[] leaf;
4    ThreadLocal<Boolean> threadSense;
5    ...
6    public void await() {
7      int me = ThreadID.get();
8      Node myLeaf = leaf[me / radix];
9      myLeaf.await();
10   }
11   ...
12 }
```

Figure 17.5 The `TreeBarrier` class: each thread indexes into an array of leaf nodes and calls that leaf's `await()` method.

barriers, and have threads combine requests going up the tree and distribute notifications going down the tree. As shown in Fig. 17.5, a *tree barrier* is characterized by a *size n*, the total number of threads, and a *radix r*, each node's number of children. For convenience, we assume there are exactly $n = r^{d+1}$ threads, where d is the depth of the tree.

Specifically, the combining tree barrier is a tree of *nodes*, where each node has a counter and a sense, just as in the sense-reversing barrier. A node's implementation is shown in Fig. 17.6. Thread i starts at leaf node $\lfloor i/r \rfloor$. The node's `await()` method is similar to the sense-reversing barrier's `await()`, the principal difference being that the last thread to arrive, the one that completes the barrier, visits the parent barrier before waking up the other threads. When r threads have arrived at the root, the barrier is complete, and the sense is reversed. As before, thread-local Boolean sense values allow the barrier to be reused without reinitialization.

The tree-structured barrier reduces memory contention by spreading memory accesses across multiple barriers. It may or may not reduce latency, depending on whether it is faster to decrement a single location or to visit a logarithmic number of barriers.

The root node, once its barrier is complete, lets notifications percolate down the tree. This approach may be good for a NUMA architecture, but it may cause nonuniform notification times. Because threads visit an unpredictable sequence of locations as they move up the tree, this approach may not work well on cacheless NUMA architectures.

Pragma 17.4.1. Tree nodes are declared as an *inner class* of the tree barrier class, so nodes are not accessible outside the class. As shown in Fig. 17.7, the tree is initialized by a recursive `build()` method. The method takes a parent node and a depth. If the depth is nonzero, it creates *radix* children, and recursively creates the children's children. If the depth is zero, it places

```
1   private class Node {
2     AtomicInteger count;
3     Node parent;
4     volatile boolean sense;
5     public Node() {
6       sense = false;
7       parent = null;
8       count = new AtomicInteger(radix);
9     }
10    public Node(Node myParent) {
11      this();
12      parent = myParent;
13    }
14    public void await() {
15      boolean mySense = threadSense.get();
16      int position = count.getAndDecrement();
17      if (position == 1) {  // I'm last
18        if (parent != null) { // Am I root?
19          parent.await();
20        }
21        count.set(radix);
22        sense = mySense;
23      } else {
24        while (sense != mySense) {};
25      }
26      threadSense.set(!mySense);
27    }
28  }
29 }
```

Figure 17.6 The `TreeBarrier` class: internal tree node.

each node in a `leaf[]` array. When a thread enters the barrier, it uses this array to choose a leaf to start from. See Appendix A.2.1 for a more complete discussion of inner classes in Java.

17.5 Static Tree Barrier

The barriers seen so far either suffer from contention (the simple and sense-reversing barriers) or have excessive communication (the combining-tree barrier). In the last barrier, threads traverse an unpredictable sequence of nodes, which makes it difficult to lay out the barriers on cacheless NUMA architectures. Surprisingly, there is another simple barrier that allows both static layout and low contention.

```
1  public class TreeBarrier implements Barrier {
2    int radix;
3    Node[] leaf;
4    int leaves;
5    ThreadLocal<Boolean> threadSense;
6    public TreeBarrier(int n, int r) {
7      radix = r;
8      leaves = 0;
9      leaf = new Node[n / r];
10     int depth = 0;
11     threadSense = new ThreadLocal<Boolean>() {
12       protected Boolean initialValue() { return true; };
13     };
14     // compute tree depth
15     while (n > 1) {
16       depth++;
17       n = n / r;
18     }
19     Node root = new Node();
20     build(root, depth - 1);
21   }
22   // recursive tree constructor
23   void build(Node parent, int depth) {
24     if (depth == 0) {
25       leaf[leaves++] = parent;
26     } else {
27       for (int i = 0; i < radix; i++) {
28         Node child = new Node(parent);
29         build(child, depth - 1);
30       }
31     }
32   }
33   ...
34 }
```

Figure 17.7 The `TreeBarrier` class: initializing a combining tree barrier. The `build()` method creates r children for each node, and then recursively creates the children's children. At the bottom, it places leaves in an array.

The static tree barrier of Fig. 17.8 works as follows. Each thread is assigned to a node in a tree (see Fig. 17.9). The thread at a node waits until all nodes below it in the tree have finished, and then informs its parent. It then spins waiting for the global sense bit to change. Once the root learns that its children are done, it toggles the global sense bit to notify the waiting threads that all threads are done. On a cache-coherent multiprocessor, completing the barrier requires $\log(n)$ steps moving up the tree, while notification simply requires changing the global sense, which is propagated by the cache-coherence mechanism. On machines without coherent caches threads propagate notification down the tree as in the combining barrier we saw earlier.

```
1  public class StaticTreeBarrier implements Barrier {
2    int radix;
3    boolean sense;
4    Node[] node;
5    ThreadLocal<Boolean> threadSense;
6    int nodes;
7    public StaticTreeBarrier(int size, int myRadix) {
8      radix = myRadix;
9      nodes = 0;
10     node = new Node[size];
11     int depth = 0;
12     while (size > 1) {
13       depth++;
14       size = size / radix;
15     }
16     build(null, depth);
17     sense = false;
18     threadSense = new ThreadLocal<Boolean>() {
19       protected Boolean initialValue() { return !sense; };
20     };
21   }
22   // recursive tree constructor
23   void build(Node parent, int depth) {
24     if (depth == 0) {
25       node[nodes++] = new Node(parent, 0);
26     } else {
27       Node myNode = new Node(parent, radix);
28       node[nodes++] = myNode;
29       for (int i = 0; i < radix; i++) {
30         build(myNode, depth - 1);
31       }
32     }
33   }
34   public void await() {
35     node[ThreadID.get()].await();
36   }
37 }
```

Figure 17.8 The `StaticTreeBarrier` class: each thread indexes into a statically assigned tree node and calls that node's `await()` method.

17.6 Termination Detecting Barriers

All the barriers considered so far were directed at computations organized in phases, where each thread finishes the work for a phase, reaches the barrier, and then starts a new phase.

There is, however, another interesting class of programs, in which each thread finishes its own part of the computation, only to be put to work again when another thread generates new work. An example of such a program is the

```
1   public Node(Node myParent, int count) {
2     children = count;
3     childCount = new AtomicInteger(count);
4     parent = myParent;
5   }
6   public void await() {
7     boolean mySense = threadSense.get();
8     while (childCount.get() > 0) {};
9     childCount.set(children);
10    if (parent != null) {
11      parent.childDone();
12      while (sense != mySense) {};
13    } else {
14      sense = !sense;
15    }
16    threadSense.set(!mySense);
17  }
18  public void childDone() {
19    childCount.getAndDecrement();
20  }
```

Figure 17.9 The `StaticTreeBarrier` class: internal `Node` class.

simplified work stealing executer pool from Chapter 16 (see Fig. 17.10). Here, once a thread exhausts the tasks in its local queue, it tries to steal work from other threads' queues. The `execute()` method itself may push new tasks onto the calling thread's local queue. Once all threads have exhausted all tasks in their queues, the threads will run forever while repeatedly attempting to steal items. Instead, we would like to devise a *termination detection* barrier so that these threads can all terminate once they have finished all their tasks.

Each thread is either *active* (it has a task to execute) or *inactive* (it has none). Note that any inactive thread may become active as long as some other thread is active, since an inactive thread may steal a task from an active one. Once all threads have become inactive, then no thread will ever become active again. Detecting that the computation as a whole has terminated is the problem of determining that at some instant in time there are no longer any active threads.

None of the barrier algorithms studied so far can solve this problem. Termination cannot be detected by having each thread announce that it has become inactive, and simply count how many have done so, because threads may repeatedly change from inactive to active and back. For example, consider threads *A*, *B*, and *C* running as shown in Fig. 17.10, and assume that each has a Boolean value indicating whether it is active or inactive. When *A* becomes inactive, it may then observe that *B* is also inactive, and then observe that *C* is inactive. Nevertheless, *A* cannot conclude that the overall computation has completed; as *B* might have stolen work from *C* after *A* checked *B*, but before it checked *C*.

```
1  public class WorkStealingThread {
2    DEQueue[] queue;
3    int size;
4    Random random;
5    public WorkStealingThread(int n) {
6      queue = new DEQueue[n];
7      size = n;
8      random = new Random();
9      for (int i = 0; i < n; i++) {
10       queue[i] = new DEQueue();
11     }
12   }
13   public void run() {
14     int me = ThreadID.get();
15     Runnable task = queue[me].popBottom();
16     while (true) {
17       while (task != null) {
18         task.run();
19         task = queue[me].popBottom();
20       }
21       while (task == null) {
22         int victim = random.nextInt() % size;
23         if (!queue[victim].isEmpty()) {
24           task = queue[victim].popTop();
25         }
26       }
27     }
28   }
29 }
```

Figure 17.10 Work stealing executer pool revisited.

```
1  public interface TDBarrier {
2    void setActive(boolean state);
3    boolean isTerminated();
4  }
```

Figure 17.11 Termination detection barrier interface.

A *termination-detection* barrier (Fig. 17.11) provides methods `setActive`(*v*) and `isTerminated`(). Each thread calls `setActive`(*true*) to notify the barrier when it becomes active, and `setActive`(*false*) to notify the barrier when it becomes inactive. The `isTerminated`() method returns *true* if and only if all threads had become inactive at some earlier instant. Fig. 17.12 shows a simple implementation of a termination-detection barrier.

The barrier encompasses an `AtomicInteger` initialized to 0. Each thread that becomes active increments the counter (Line 8) and each thread that becomes inactive decrements it (Line 10). The computation is deemed to have terminated when the counter reaches zero (Line 14).

```
1  public class SimpleTDBarrier implements TDBarrier {
2    AtomicInteger count;
3    public SimpleTDBarrier(int n){
4      count = new AtomicInteger(n);
5    }
6    public void setActive(boolean active) {
7      if (active) {
8        count.getAndIncrement();
9      } else {
10       count.getAndDecrement();
11     }
12   }
13   public boolean isTerminated() {
14     return count.get() == 0;
15   }
16 }
```

Figure 17.12 A simple termination detecting barrier.

```
1    public void run() {
2      int me = ThreadID.get();
3      tdBarrier.setActive(true);
4      Runnable task = queue[me].popBottom();
5      while (true) {
6        while (task != null) {
7          task.run();
8          task = queue[me].popBottom();
9        }
10       tdBarrier.setActive(false);
11       while (task == null) {
12         int victim = random.nextInt() % queue.length;
13         if (!queue[victim].isEmpty()) {
14           tdBarrier.setActive(true);
15           task = queue[victim].popTop();
16           if (task == null) {
17             tdBarrier.setActive(false);
18           }
19         }
20         if (tdBarrier.isTerminated()) {
21           return;
22         }
23       }
24     }
25   }
26 }
```

Figure 17.13 Work-stealing executer pool: the `run()` method with termination.

The termination-detection barrier works only if used correctly. Fig. 17.13 shows how to modify the work-stealing thread's `run()` method to return when the computation has terminated. Initially, every thread registers as active (Line 3). Once a thread has exhausted its local queue, it registers as inactive (Line 10).

Before it tries to steal a new task, however, it must register as active (Line 14). If the theft fails, it registers as inactive again (Line 17).

Notice that a thread sets its state to active before stealing a task. Otherwise, if a thread were to steal a task while inactive, then the thread whose task was stolen might also declare itself inactive, resulting in a computation where all threads declare themselves inactive while the computation continues.

Here is a subtle point. A thread tests whether the queue is empty (Line 13) before it attempts to steal a task. This way, it avoids declaring itself active if there is no chance the theft will succeed. Without this precaution, it is possible that the threads will not detect termination because each one repeatedly switches to an active state before a steal attempt that is doomed to fail.

Correct use of the termination-detection barrier must satisfy both a safety and a liveness property. The safety property is that if `isTerminated`() returns *true*, then the computation really has terminated. Safety requires that no active thread ever declare itself inactive, because it could trigger an incorrect termination detection. For example, the work-stealing thread of Fig. 17.13 would be incorrect if the thread declared itself to be active only after successfully stealing a task. By contrast, it is safe for an inactive thread to declare itself active, which may occur if the thread is unsuccessful in stealing work at Line 15.

The liveness property is that if the computation terminates, then `isTerminated`() eventually returns *true*. (It is not necessary that termination be detected instantly.) While safety is not jeopardized if an inactive thread declares itself active, liveness will be violated if a thread that does not succeed in stealing work fails to declare itself inactive again (Line 15), because termination will not be detected when it occurs.

17.7 Chapter Notes

John Mellor–Crummey and Michael Scott [113] provide a survey of several barrier algorithms, though the performance numbers they provide should be viewed from a historical perspective. The combining tree barrier is based on code due to John Mellor–Crummey and Michael Scott [113], which is in turn based on the combining tree algorithm of Pen-Chung Yew, Nian-Feng Tzeng, and Duncan Lawrie [151]. The dissemination barrier is credited to Debra Hensgen, Raphael Finkel, and Udi Manber [59]. The tournament tree barrier used in the exercises is credited to John Mellor–Crummey and Michael Scott [113]. The simple barriers and the static tree barrier are most likely folklore. We learned of the static tree barrier from Beng-Hong Lim. The termination detection barrier and its application to an executer pool are based on a variation suggested by Peter Kessler to an algorithm by Dave Detlefs, Christine Flood, Nir Shavit, and Xiolan Zhang [41].

17.8 Exercises

Exercise 198. Fig. 17.14 shows how to use barriers to make a parallel prefix computation work on an asynchronous architecture.

A *parallel prefix* computation, given a sequence $a_0, \ldots, a_{m-1}$, of numbers, computes in parallel the partial sums:

$$b_i = \sum_{j=0}^{i} a_j.$$

In a synchronous system, where all threads take steps at the same time, there are simple, well-known algorithms for m threads to compute the partial sums in $\log m$ steps. The computation proceeds in a sequence of rounds, starting at round zero. In round r, if $i \geqslant 2^r$, thread i reads the value at $a[i - 2^r]$ into a local variable. Next, it adds that value to $a[i]$. Rounds continue until $2^r \geqslant m$. It is not hard to see that after $\log_2(m)$ rounds, the array a contains the partial sums.

1. What could go wrong if we executed the parallel prefix on $n > m$ threads?
2. Modify this program, adding one or more barriers, to make it work properly in a concurrent setting with n threads. What is the minimum number of barriers that are necessary?

Exercise 199. Change the sense-reversing barrier implementation so that waiting threads call `wait()` instead of spinning.

```
1  class Prefix extends java.lang.Thread {
2    private int[] a;
3    private int i;
4    public Prefix(int[] myA, int myI) {
5      a = myA;
6      i = myI;
7    }
8    public void run() {
9      int d = 1, sum = 0;
10     while (d < m) {
11       if (i >= d)
12         sum = a[i-d];
13       if (i >= d)
14         a[i] += sum;
15       d = d * 2;
16     }
17   }
18 }
```

Figure 17.14 Parallel prefix computation.

- Give an example of a situation where suspending threads is better than spinning.
- Give an example of a situation where the other choice is better.

Exercise 200. Change the tree barrier implementation so that it takes a `Runnable` object whose `run()` method is called once after the last thread arrives at the barrier, but before any thread leaves the barrier.

Exercise 201. Modify the combining tree barrier so that nodes can use any barrier implementation, not just the sense-reversing barrier.

Exercise 202. A *tournament tree barrier* (Class `TourBarrier` in Fig. 17.15) is an alternative tree-structured barrier. Assume there are n threads, where n is a power of 2. The tree is a binary tree consisting of $2n - 1$ nodes. Each leaf is owned by a single, statically determined thread. Each node's two children are linked as *partners*. One partner is statically designated as *active*, and the other as *passive*. Fig. 17.16 illustrates the tree structure.

```
1     private class Node {
2      volatile boolean flag;   // signal when done
3      boolean active;          // active or passive?
4      Node parent;             // parent node
5      Node partner;            // partner node
6      // create passive node
7      Node() {
8        flag   = false;
9        active = false;
10       partner = null;
11       parent = null;
12     }
13     // create active node
14     Node(Node myParent) {
15       this();
16       parent = myParent;
17       active = true;
18     }
19     void await(boolean sense) {
20       if (active) { // I'm active
21        while (flag != sense) {};                  // wait for partner
22        if (parent != null) {parent.await(sense)}; // wait for parent
23        partner.flag = sense;                      // tell partner
24        } else {                      // I'm passive
25          partner.flag = sense;       // tell partner
26          while (flag != sense) {};   // wait for partner
27       }
28     }
29   }
```

Figure 17.15 The `TourBarrier` class.

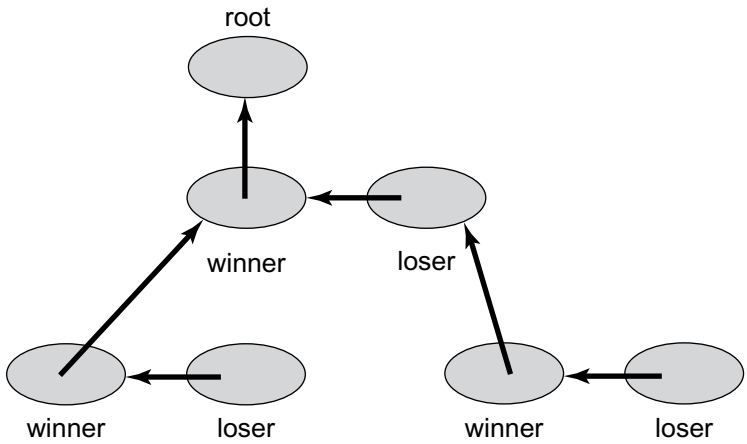


Figure 17.16 The `TourBarrier` class: information flow. Nodes are paired statically in active/passive pairs. Threads start at the leaves. Each thread in an active node waits for its passive partner to show up; then it proceeds up the tree. Each passive thread waits for its active partner for notification of completion. Once an active thread reaches the root, all threads have arrived, and notifications flow down the tree in the reverse order.

Each thread keeps track of the current sense in a thread-local variable. When a thread arrives at a passive node, it sets its active partner's `sense` field to the current sense, and spins on its own `sense` field until its partner changes that field's value to the current sense. When a thread arrives at an active node, it spins on its `sense` field until its passive partner sets it to the current sense. When the field changes, that particular barrier is complete, and the active thread follows the `parent` reference to its parent node. Note that an active thread at one level may become passive at the next level. When the root node barrier is complete, notifications percolate down the tree. Each thread moves back down the tree setting its partner's `sense` field to the current sense.

This barrier improves a little on the combining tree barrier of Fig. 17.5. Explain how.

The tournament barrier code uses `parent` and `partner` references to navigate the tree. We could save space by eliminating these fields and keeping all the nodes in a single array with the root at index 0, the root's children at indexes 1 and 2, the grandchildren at indexes 3–6, and so on. Re-implement the tournament barrier to use indexing arithmetic instead of references to navigate the tree.

Exercise 203. The combining tree barrier uses a single thread-local sense field for the entire barrier. Suppose instead we were to associate a thread-local sense with each node as in Fig. 17.8. Either:

- Explain why this implementation is equivalent to the other one, except that it consumes more memory, or.
- Give a counterexample showing that this implementation is incorrect.

```
1  private class Node {
2    AtomicInteger count;
3    Node parent;
4    volatile boolean sense;
5    int d;
6    // construct root node
7    public Node() {
8      sense = false;
9      parent = null;
10     count = new AtomicInteger(radix);
11     ThreadLocal<Boolean> threadSense;
12     threadSense = new ThreadLocal<Boolean>() {
13       protected Boolean initialValue() { return true; };
14   };
15   }
16   public Node(Node myParent) {
17     this();
18     parent = myParent;
19   }
20   public void await() {
21     boolean mySense = threadSense.get();
22     int position = count.getAndDecrement();
23     if (position == 1) {     // I'm last
24       if (parent != null) { // root?
25         parent.await();
26       }
27       count.set(radix);   // reset counter
28       sense = mySense;
29     } else {
30       while (sense != mySense) {};
31     }
32     threadSense.set(!mySense);
33   }
34 }
```

Figure 17.17 Thread-local tree barrier.

Exercise 204. The tree barrier works "bottom-up," in the sense that barrier completion moves from the leaves up to the root, while wake-up information moves from the root back down to the leaves. Figs. 17.18 and 17.19 show an alternative design, called a *reverse tree barrier*, which works just like a tree barrier except for the fact that barrier completion starts at the root and moves down to the leaves. Either:

- Sketch an argument why this is correct, perhaps by reduction to the standard tree barrier, or
- Give a counterexample showing why it is incorrect.

Exercise 205. Implement an n-thread reusable barrier from an n-wire counting network and a single Boolean variable. Sketch a proof that the barrier works.

```
1  public class RevBarrier implements Barrier {
2    int radix;
3    ThreadLocal<Boolean> threadSense;
4    int leaves;
5    Node[] leaf;
6    public RevBarrier(int mySize, int myRadix) {
7      radix = myRadix;
8      leaves = 0;
9      leaf = new Node[mySize / myRadix];
10     int depth = 0;
11     threadSense = new ThreadLocal<Boolean>() {
12       protected Boolean initialValue() { return true; };
13     };
14     // compute tree depth
15     while (mySize > 1) {
16       depth++;
17       mySize = mySize / myRadix;
18     }
19     Node root = new Node();
20     root.d = depth;
21     build(root, depth - 1);
22   }
23   // recursive tree constructor
24   void build(Node parent, int depth) {
25     // are we at a leaf node?
26     if (depth == 0) {
27       leaf[leaves++] = parent;
28     } else {
29       for (int i = 0; i < radix; i++) {
30         Node child = new Node(parent);
31         child.d = depth;
32         build(child, depth - 1);
33       }
34     }
35   }
```

Figure 17.18 Reverse tree barrier Part 1.

Exercise 206. Can you devise a "distributed" termination detection algorithm for the executer pool in which threads do not repeatedly update or test a central location for termination, but rather use only local uncontended variables? Variables may be unbounded, but state changes should take constant time, (so you cannot parallelize the shared counter).

Hint: adapt the atomic snapshot algorithm from Chapter 4.

Exercise 207. A *dissemination barrier* is a symmetric barrier implementation in which threads spin on statically-assigned locally-cached locations using only loads and stores. As illustrated in Fig. 17.20, the algorithm runs in a series of rounds. At round r, thread i notifies thread $i + 2^r \pmod n$, (where n is the number of threads) and waits for notification from thread $i - 2^r \pmod n$.

```
36    public void await() {
37      int me = ThreadInfo.getIndex();
38      Node myLeaf = leaf[me / radix];
39      myLeaf.await(me);
40    }
41    private class Node {
42      AtomicInteger count;
43      Node parent;
44      volatile boolean sense;
45      int d;
46      // construct root node
47      public Node() {
48        sense = false;
49        parent = null;
50        count = new AtomicInteger(radix);
51      }
52      public Node(Node myParent) {
53        this();
54        parent = myParent;
55      }
56      public void await(int me) {
57        boolean mySense = threadSense.get();
58        // visit parent first
59        if ((me % radix) == 0) {
60          if (parent != null) { // root?
61            parent.await(me / radix);
62          }
63        }
64        int position = count.getAndDecrement();
65        if (position == 1) {  // I'm last
66          count.set(radix);   // reset counter
67          sense = mySense;
68        } else {
69          while (sense != mySense) {};
70        }
71        threadSense.set(!mySense);
72      }
73    }
74  }
```

Figure 17.19 Reverse tree barrier Part 2: correct or not?

For how many rounds must this protocol run to implement a barrier? What if n is not a power of 2? Justify your answers.

Exercise 208. Give a reusable implementation of a dissemination barrier in Java.

Hint: you may want to keep track of both the parity and the sense of the current phase.

Exercise 209. Create a table that summarizes the total number of operations in the static tree, combining tree, and dissemination barriers.

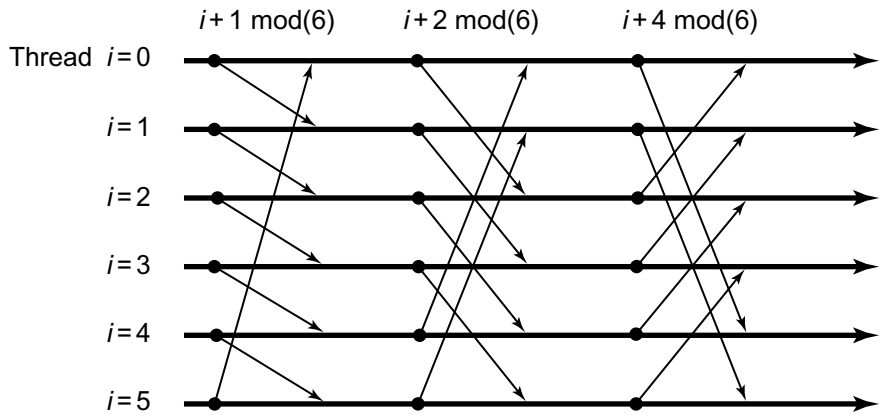


Figure 17.20 Communication in the dissemination barrier. In each round r a thread i communicates with thread $i + 2^r \pmod{n}$.

Exercise 210. In the termination detection barrier, the state is set to active before stealing the task; otherwise the stealing thread could be declared inactive; then it would steal a task, and before setting its state back to active, the thread it stole from could become inactive. This would lead to an undesirable situation in which all threads are declared inactive yet the computation continues. Can you devise a terminating executer pool in which the state is set to active only *after* successfully stealing a task?